

“Campus Voice”

A Project Report Submitted in the partial fulfillment of the requirements for the
award of the degree of

BACHELOR OF TECHNOLOGY

in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

by

S. SAHASRA :2520030008

P. TEJASWINI :2520030011

V. SIRI CHANDANA :2520030609

Under the Guidance of

Dr. S. Vinay Kumar

Associate Professor

Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

2026-27

Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

Declaration

The Project Report entitled “Campus Voice” is a record of Bonafide work of S. Sahasra – 2520030008, P. Tejaswini – 2520030011, V. Siri Chandana – 2520030609 submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering. The results embodied in this report have not been copied from any other departments/University/Institute.

S. SAHASRA – 2520030008

P. TEJASWINI – 2520030011

V. SIRI CHANDANA – 2520030609

Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

CERTIFICATE

This is to certify that the mini project based report entitled “Campus Voice” is a bonafide work done and submitted by S. Sahasra – 2520030008, P. Tejaswini – 2520030011, V. Siri Chandana – 2520030609 in partial fulfillment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in Department of Computer Science Engineering, during the academic year 2026-27.

Signature of the Guide

(Dr. S. Vinay Kumar)

Signature of the HOD

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide, Dr. S. Vinay Kumar, Associate Professor, Department of Computer Science and Engineering, for his valuable guidance, constant encouragement and timely suggestions throughout the development of our project, “Campus Voice – a terminal-based student feedback and voting system”. His support helped us to understand the requirements of the project and to present our work in a structured manner.

We are thankful to Dr. P. Venkateshwara Rao, Associate Professor and Head of the Department of Computer Science and Engineering, for providing the facilities and the supportive academic environment that enabled us to complete this project successfully. We also thank all the faculty members of the department for their cooperation and encouragement.

Finally, we thank our parents, friends and all those who directly or indirectly helped us during the course of this work for their continuous support and motivation.

S. SAHASRA – 2520030008

P. TEJASWINI – 2520030011

V. SIRI CHANDANA – 2520030609

Abstract

Campus Voice is a terminal-based student feedback and voting system developed in Java to collect, organize, search, compare, and prioritize campus-related feedback. The project is implemented without a web frontend, backend server, API layer, or database. Instead, it uses the Java console for user interaction and text files for persistent storage. Feedback records are stored as individual files in the corpus directory, while voting and feedback-status information are maintained in text files under the data directory.

The system provides two console modules: Student and Admin. The Student Module allows users to submit feedback, view top-voted feedback, search the feedback corpus, and upvote an existing concern. When new feedback is submitted, it is compared against all existing feedback using Levenshtein similarity. A similarity threshold of 60% is used to identify a potentially duplicate concern. If a related record is found, the user is shown the existing feedback and can upvote it instead of creating a duplicate record.

The project also demonstrates several Data Structures and Algorithms concepts. KMP string matching is used for keyword search, Levenshtein edit distance is used for similarity-based duplicate recognition, greedy maximum coverage is used for priority analysis, bipartite matching is used to assign pending feedback to review slots, and reservoir sampling is used for random feedback auditing. These algorithms make the project suitable for demonstrating practical DSA applications in a campus feedback scenario.

Testing was performed through the terminal using valid inputs, invalid inputs, duplicate feedback, keyword searches, voting operations, status updates, priority analysis, review scheduling, and random audit sampling. The final system provides a lightweight, file-based, terminal-driven implementation that demonstrates how DSA techniques can be integrated into a practical student feedback management problem.

Table of Contents

Section	Page
1. INTRODUCTION	6
1.1 Overview	6
1.2 Problem Statement	7
1.3 Scope of the Project	7
1.4 Objectives	8
2. SYSTEM REQUIREMENTS	10
2.1 Functional Requirements	10
2.2 Non-Functional Requirements	11
3. SYSTEM DESIGN	12
3.1 High-Level System Architecture	12
3.2 Components of the System	13
3.3 File-Based Data Organization	14
3.4 Overall Working Flow	15
4. IMPLEMENTATION	16
4.1 Module-Wise Implementation	16
4.2 Feedback Submission and Similarity Detection	16
4.3 KMP Keyword Search	17
4.4 Voting and Duplicate-Vote Prevention	18
4.5 Administrator Module	19
4.6 Priority Analysis	19
4.7 Feedback Review Scheduling	19
4.8 Random Feedback Audit	20
4.9 File Handling and Corpus Management	21
4.10 Pseudocode	21

4.11 Error Handling and Data Integrity	23
5. RESULTS AND TESTING	24
5.1 Main Menu Execution	24
5.2 Student Module Execution	24
5.3 Similar Feedback Detection	25
5.4 Keyword Search	25
5.5 Voting	26
5.6 Administrator Operations	26
5.7 Overall Test Results	26
6. CHALLENGES AND LIMITATIONS	27
6.1 Challenges Faced	27
6.2 Solutions Applied	27
6.3 Limitations	27
7. CONCLUSION AND FUTURE WORK	29
7.1 Conclusion	29
7.2 Future Work	29
7.3 Skills Learned During Development	30
REFERENCES	31
APPENDIX A – SETUP AND EXECUTION	32
APPENDIX B – IMPORTANT CODE SNIPPETS	33

List of Figures

Figure	Description
Fig. 1.1	Campus Voice – Terminal-Based Feedback System Page 7
Fig. 2.1	Functional Modules Page 11
Fig. 3.1	Detailed System Architecture Page 12
Fig. 3.2	Components of Campus Voice Page 13
Fig. 3.3	File-Based Data Organization Page 14
Fig. 3.4	Overall Working Flow Page 15
Fig. 4.1	Feedback Submission and Similarity Detection Page 17
Fig. 4.2	KMP Keyword Search Page 18
Fig. 4.3	Voting and Duplicate-Vote Prevention Page 18
Fig. 4.4	Administrator DSA Operations Page 19
Fig. 4.5	Feedback Review Scheduling Page 20
Fig. 4.6	Reservoir Sampling for Random Audit Page 21
Fig. 5.1	Representative Terminal Execution – Main Menu Page 24
Fig. 5.2	Representative Terminal Execution – Duplicate Detection Page 25
Fig. 5.3	Representative Terminal Execution – Search Page 25

1. INTRODUCTION

1.1 Overview

Student feedback is an important source of information for understanding academic, infrastructure, hostel, transport, connectivity, and other campus-related concerns. In a conventional process, similar complaints may be submitted repeatedly, making it difficult to identify which concerns are common and which require attention first. A lightweight feedback system can reduce this repetition by organizing feedback into identifiable records and allowing students to support an existing concern.

Campus Voice addresses this problem through a Java-based terminal application. The program starts from Main.java and presents a console menu from which the user can enter either the Student Module or the Admin Module. All interaction takes place through standard input and output. There is no browser interface or web server in the current implementation.

The feedback corpus is maintained as text files named Feedback01.txt, Feedback02.txt and so on. The current project corpus contains 99 feedback files, covering IDs 1–20 and 22–100. The missing ID 21 does not affect the program because new feedback IDs are generated by finding the current maximum ID and adding one. Voting information is stored in data/votes.txt and feedback status information is stored in data/status.txt.

Campus Voice - Terminal-Based Feedback System

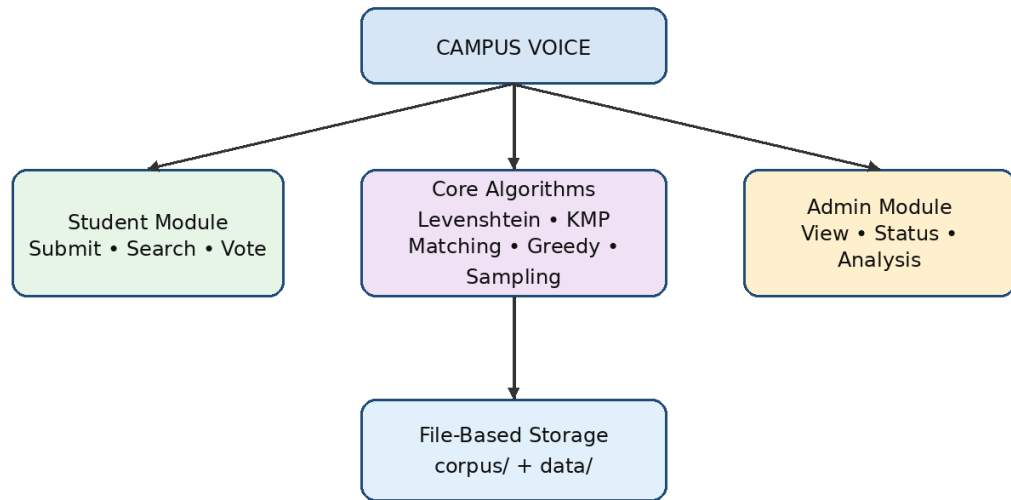


Fig. 1.1 Campus Voice – Terminal-Based Feedback System

1.2 Problem Statement

The main problem addressed by Campus Voice is the management of repeated and difficult-to-prioritize student feedback. When students describe the same issue using slightly different wording, treating every submission as a new record creates unnecessary duplication. In addition, a plain collection of feedback records does not directly show which issues have greater student support.

- Repeated or closely related feedback may create duplicate records.
- Students need a simple way to search existing concerns before adding another record.
- Administrators need a way to view feedback and update its status.
- Student support should be represented through voting while preventing repeated votes by the same student for the same feedback.
- Administrators need practical DSA-based tools for selecting priority issues, scheduling reviews, and auditing a sample of feedback.

Campus Voice solves these problems through similarity checking, keyword search, voting, status management, and DSA-based administrative analysis in a single terminal application.

1.3 Scope of the Project

The scope of the current project is limited to a terminal-based Java implementation. The system focuses on the feedback lifecycle from creation and storage to search, voting, prioritization, status tracking, scheduling, and random auditing. The project does not include a web or mobile frontend, a backend server, REST APIs, cloud services, or a relational database.

- Student-side feedback submission.
- Similarity-based detection of an existing feedback record using a 60% Levenshtein threshold.
- Viewing feedback sorted by vote count.
- Keyword search using KMP.
- Upvoting with duplicate-vote prevention based on Student ID and Feedback ID.
- Administrator viewing and status updates.
- Greedy priority analysis using maximum-coverage style selection.
- Bipartite matching for review-slot assignment.
- Reservoir sampling for random feedback audits.
- File-based persistence using the corpus and data directories.

1.4 Objectives

- To develop a simple terminal-based campus feedback system using Java.
- To store feedback records as individual text files with unique numeric IDs.
- To detect similar feedback using Levenshtein edit distance and a 60% similarity threshold.
- To implement efficient keyword search using the KMP algorithm.
- To provide a voting mechanism that prevents repeated votes by the same student.
- To allow administrators to update the status of feedback as Pending, In Progress, or Completed.

- To apply greedy maximum coverage for selecting priority issues.
- To use bipartite matching for assigning pending feedback to available review slots.
- To use reservoir sampling to select a random audit sample from the feedback corpus.
- To demonstrate practical applications of DSA concepts in a real-world campus problem.

1.5 Data Structures and Algorithms Mapping

Course / DSA Area	Algorithm Used	Purpose in Campus Voice
CO2	KMP String Matching	Keyword-based feedback search
CO3	Levenshtein Edit Distance	60% similarity-based duplicate recognition
CO4	Bipartite Matching	Assign pending feedback to review slots
CO5	Greedy Maximum Coverage	Select priority issues based on student coverage
CO6	Reservoir Sampling	Random feedback audit

2. SYSTEM REQUIREMENTS

The system requirements describe the functions provided by the terminal application and the quality characteristics expected from the implementation. Because the project is console-based, the requirements focus on Java execution, text-file persistence, algorithmic processing, and clear terminal interaction.

2.1 Functional Requirements

Requirement	Description
Main Menu	Provides Student, Admin, and Exit options.
Student Module	Provides Submit Feedback, Top Voted Feedback, Search Feedback, and Back.
Feedback Submission	Accepts non-empty feedback text and compares it with the complete corpus.
Similarity Detection	Computes the highest Levenshtein similarity against existing feedback.
Duplicate Handling	If similarity is at least 60%, displays the existing record and offers an upvote.
New Feedback Creation	Creates the next FeedbackXX.txt file when no sufficiently similar record exists.
Top Voted Feedback	Sorts feedback records by vote count and displays them.
Keyword Search	Uses KMP to search feedback text for a keyword.
Voting	Records Student ID and Feedback ID and prevents the same student from voting twice for the same record.
Admin View	Displays every feedback record with vote count and status.
Status Update	Changes feedback status to Pending, In Progress, or Completed.
Priority Analysis	Uses greedy maximum coverage to select a limited number of high-coverage issues.
Review Scheduling	Uses bipartite matching to assign pending feedback to available review slots.
Random Audit	Uses reservoir sampling to choose a random sample of feedback files.

Functional Modules

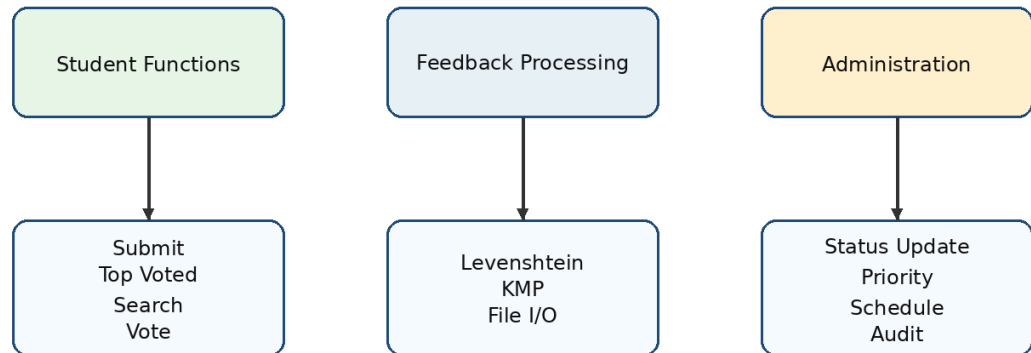


Fig. 2.1 Functional Modules

2.2 Non-Functional Requirements

Requirement	Description
Usability	Menus and prompts should be understandable to a terminal user.
Performance	Feedback search and comparison should complete within reasonable time for the stored corpus.
Reliability	Feedback, vote, and status files should be read and written consistently.
Maintainability	Student, Admin, and algorithm classes are separated into packages.
Accuracy	Similarity, search, vote validation, matching, and sampling should produce correct results according to their algorithms.
Data Integrity	A feedback record should retain its ID and text, and duplicate votes should not be added.
Portability	The project should run on a machine with a compatible Java compiler/runtime.
Extensibility	Additional algorithms or user functions can be added without requiring a web stack.

3. SYSTEM DESIGN

Campus Voice follows a simple modular console architecture. The system does not separate a web presentation layer from a backend service. Instead, Main.java provides the entry point, the StudentModule and AdminModule classes implement user operations, the algorithms package contains reusable DSA implementations, and the corpus/data folders provide file-based persistence.

3.1 High-Level System Architecture

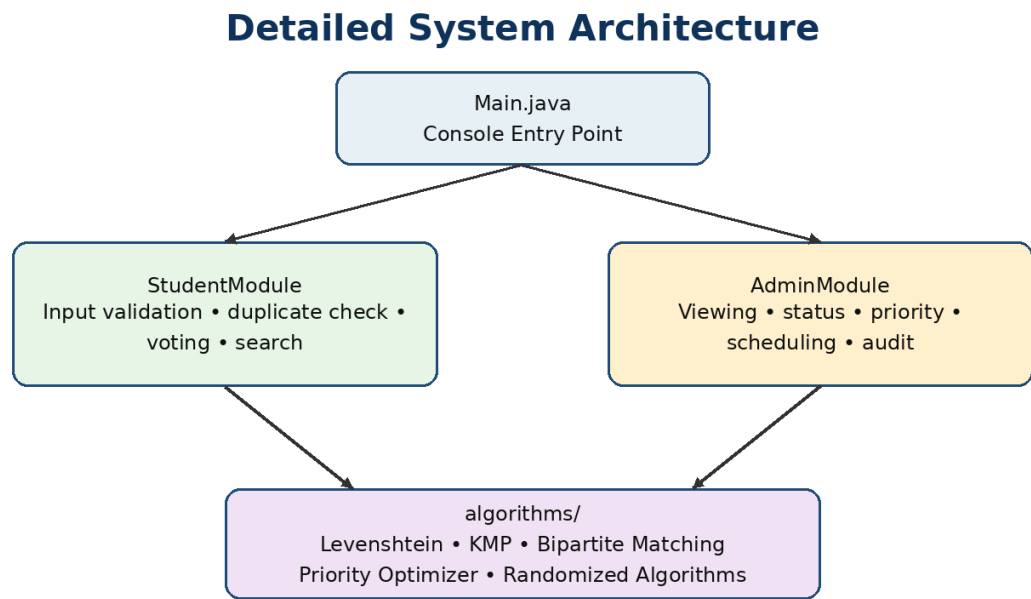


Fig. 3.1 Detailed System Architecture

The architecture begins with Main.java. It creates a Scanner for console input and routes the user to StudentModule or AdminModule. StudentModule performs student operations and uses LevenshteinDistance, KMPSearch, and ReadFileContents. AdminModule performs administration operations and uses ReadFileContents, PriorityOptimizer, BipartiteMatching, and RandomizedAlgorithms. Both modules access the text files stored in the project folders.

3.2 Components of the System

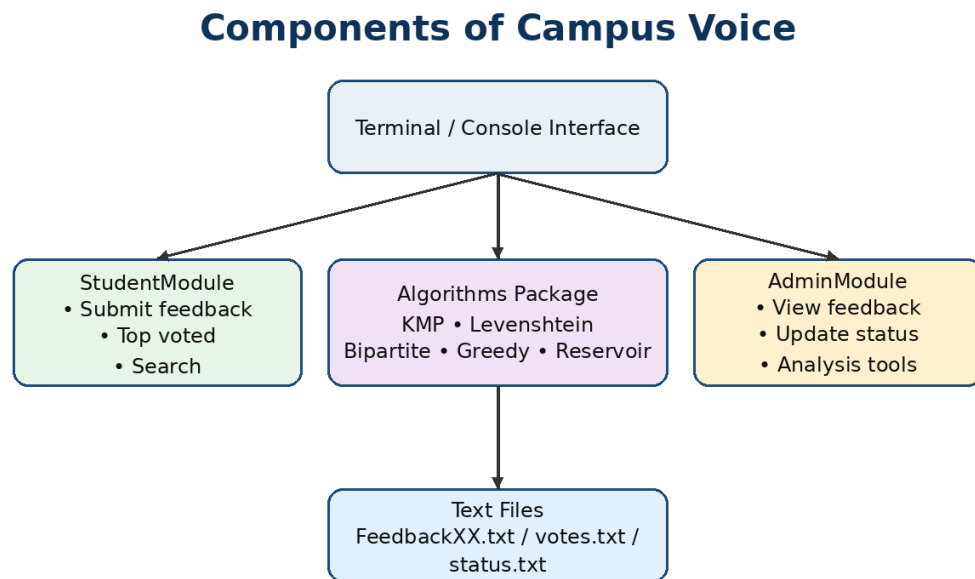


Fig. 3.2 Components of Campus Voice

3.2.1 Main Module

Main.java is the entry point of the application. It displays the CAMPUSVOICE menu, accepts a choice, and creates either a StudentModule or an AdminModule using the same Scanner object. Choosing Exit terminates the program.

3.2.2 Student Module

StudentModule.java implements feedback submission, top-voted feedback display, keyword search, voting, and vote counting. It also creates the corpus and data directories if they do not exist and ensures that votes.txt is available for use.

3.2.3 Admin Module

AdminModule.java implements feedback viewing, status updates, priority analysis, review scheduling, and random auditing. It maintains status.txt and votes.txt as the administrative data files.

3.2.4 Algorithms Package

The algorithms package contains reusable implementations of LevenshteinDistance, KMPSearch, BipartiteMatching, PriorityOptimizer, RandomizedAlgorithms, and ReadFileContents. These classes keep the DSA logic separate from the menu and file-management logic.

3.3 File-Based Data Organization

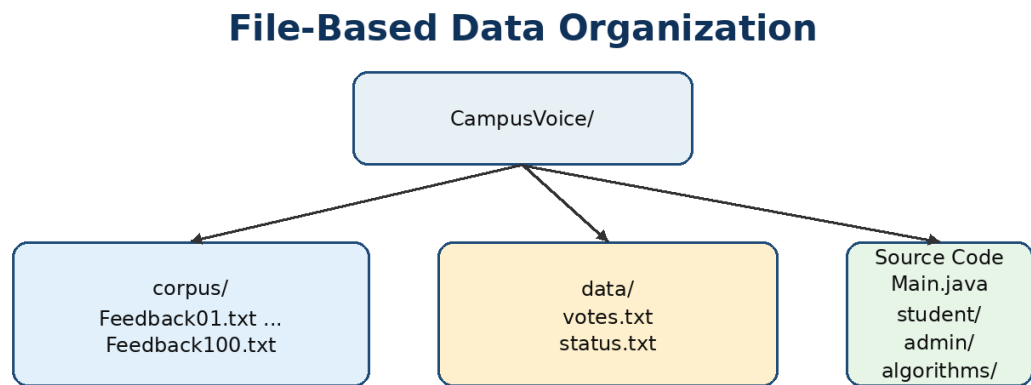


Fig. 3.3 File-Based Data Organization

Path	Purpose
corpus/Feedback01.txt ... Feedback100.txt	Stores individual feedback records as plain text.
data/votes.txt	Stores Student ID and Feedback filename pairs separated by .
data/status.txt	Stores Feedback filename and current status separated by .
Main.java	Console entry point.
student/StudentModule.java	Student-side operations.
admin/AdminModule.java	Administrator-side operations.
algorithms/*.java	DSA and file utility implementations.

3.4 Overall Working Flow

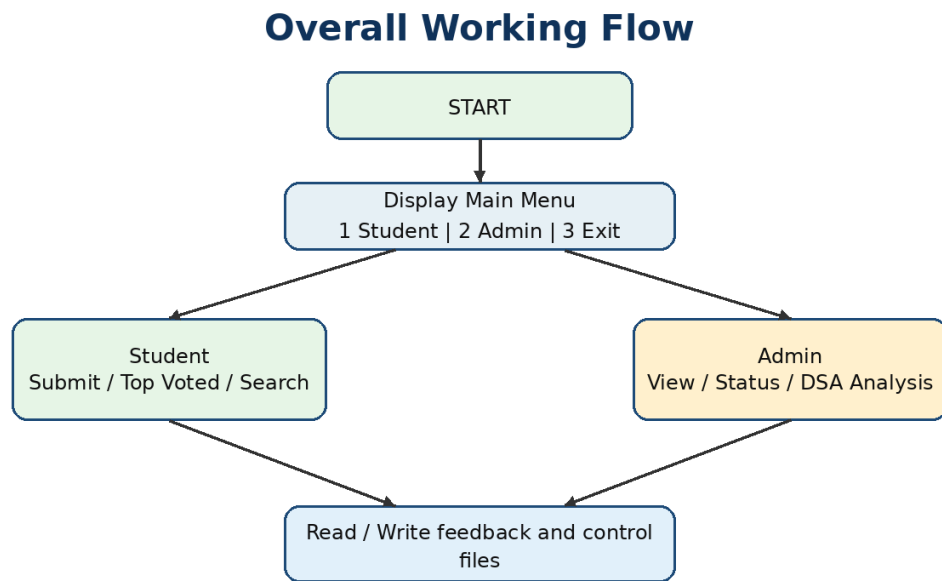


Fig. 3.4 Overall Working Flow

When the application starts, the main menu is displayed. Selecting Student opens the Student Module; selecting Admin opens the Admin Module. Each module remains active until the user chooses Back. Feedback operations read from or write to the text-file corpus and data files. This structure keeps the application simple while still demonstrating multiple algorithmic operations.

4. IMPLEMENTATION

The implementation is written in Java and is organized into packages for student operations, administrative operations, and algorithms. Compilation is performed using `javac` and execution is performed using `java`. The program is completely terminal-driven.

4.1 Module-Wise Implementation

Module / Class	Main Responsibilities
Main.java	Display main menu and route to Student/Admin modules.
StudentModule.java	Submit, search, view top-voted feedback, vote.
AdminModule.java	View feedback, update status, priority analysis, review scheduling, random audit.
LevenshteinDistance.java	Normalize feedback and calculate edit-distance similarity.
KMPSearch.java	Build LPS array and perform case-insensitive substring search.
BipartiteMatching.java	Compute maximum matching between feedback records and review slots.
PriorityOptimizer.java	Select issues using greedy maximum coverage.
RandomizedAlgorithms.java	Perform reservoir sampling for random audits.
ReadFileContents.java	Discover, sort, read, and write feedback files.

4.2 Feedback Submission and Similarity Detection

When the student selects Submit Feedback, the program reads the feedback text and rejects an empty input. It then scans every file returned by `ReadFileContents.getCorpusFiles()`. For each file, `LevenshteinDistance.similarity()` computes a normalized similarity value. The program retains the file with the highest similarity. If the highest similarity is at least 60%, the existing feedback ID and text are displayed and the user is offered an opportunity to upvote it. If the threshold is not reached, the program generates the next ID and writes the new text to `FeedbackXX.txt`.

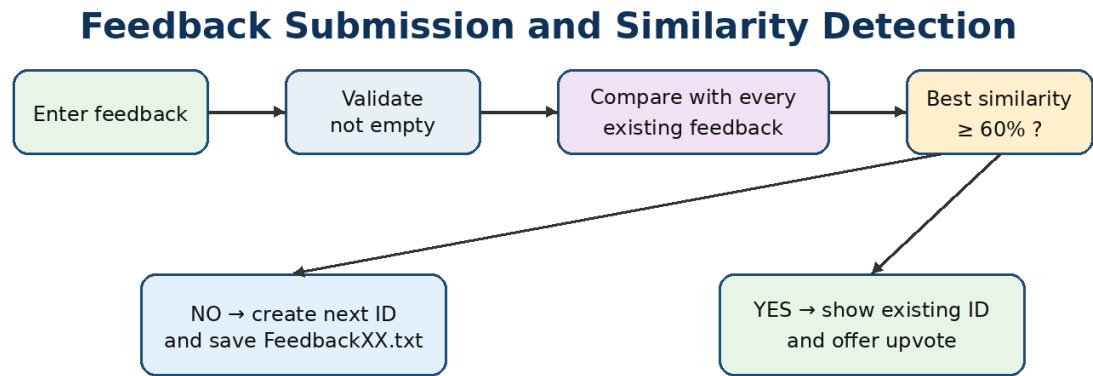


Fig. 4.1 Feedback Submission and Similarity Detection

4.2.1 Similarity Formula

The implementation first normalizes the strings by converting them to lowercase, replacing non-alphanumeric characters with spaces, collapsing repeated whitespace, and trimming the result. The Levenshtein distance is then divided by the maximum normalized string length. Similarity is calculated as:

```

similarity = 1 - (LevenshteinDistance / max(normalizedLength1,
normalizedLength2))

```

A similarity of 1.0 represents identical normalized text, while lower values represent greater textual difference. The Student Module compares the best score against the constant threshold of 60.0 percent.

4.3 KMP Keyword Search

The Search Feedback option accepts a keyword and checks every feedback file. KMPSearch first converts both text and pattern to lowercase, builds the Longest Prefix Suffix (LPS) array for the pattern, and then scans the text without restarting the pattern from the beginning after every mismatch. If the keyword occurs, the corresponding Feedback ID, vote count, and feedback text are displayed.

KMP Keyword Search

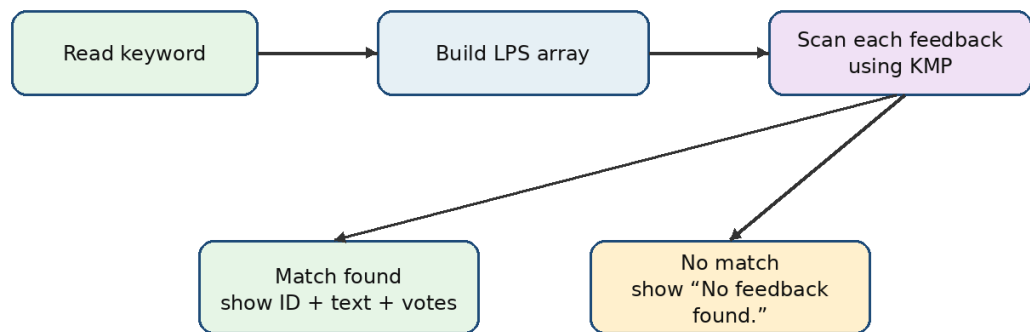


Fig. 4.2 KMP Keyword Search

4.4 Voting and Duplicate-Vote Prevention

A student can upvote an existing feedback record either after similarity detection or from the Top Voted Feedback list. The program asks for a Student ID. Before appending a vote, it scans votes.txt and checks whether the same Student ID is already associated with the same Feedback filename. If such a pair exists, the vote is rejected. Otherwise, the pair is appended to votes.txt.

Voting and Duplicate-Vote Prevention

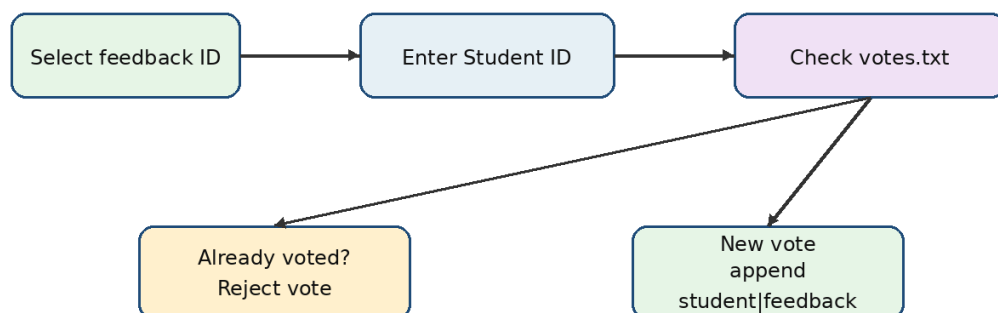


Fig. 4.3 Voting and Duplicate-Vote Prevention

4.5 Administrator Module

The Admin Module is a terminal menu with five operations: View All Feedback, Update Feedback Status, Priority Analysis, Feedback Review Schedule, and Random Feedback Audit. View All Feedback reads the complete corpus and prints each Feedback ID, vote count, status, and text. Status Update changes the corresponding entry in status.txt to Pending, In Progress, or Completed.

4.6 Priority Analysis

Priority Analysis reads votes.txt and builds a mapping from each feedback record to the set of students who voted for it. Each feedback record is represented as an Issue object containing its filename, student set, and upvote count. PriorityOptimizer then repeatedly selects the remaining issue that covers the largest number of students not already covered by previously selected issues. If the coverage gain is tied, the issue with more upvotes is preferred.

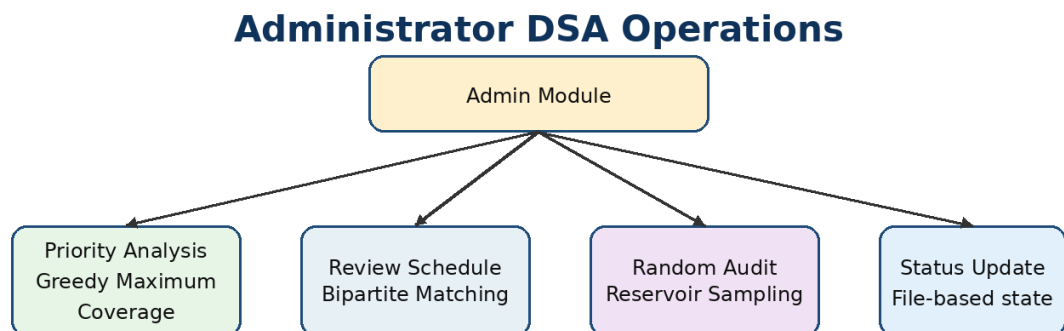


Fig. 4.4 Administrator DSA Operations

4.7 Feedback Review Scheduling

Review scheduling first collects all feedback whose current status is Pending. The administrator enters the number of available review slots and names the slots. The implementation creates a bipartite graph in which every pending feedback is connected to every available slot. BipartiteMatching applies an augmenting-path

maximum matching method so that each slot receives at most one feedback record. The generated assignment is printed in the terminal.

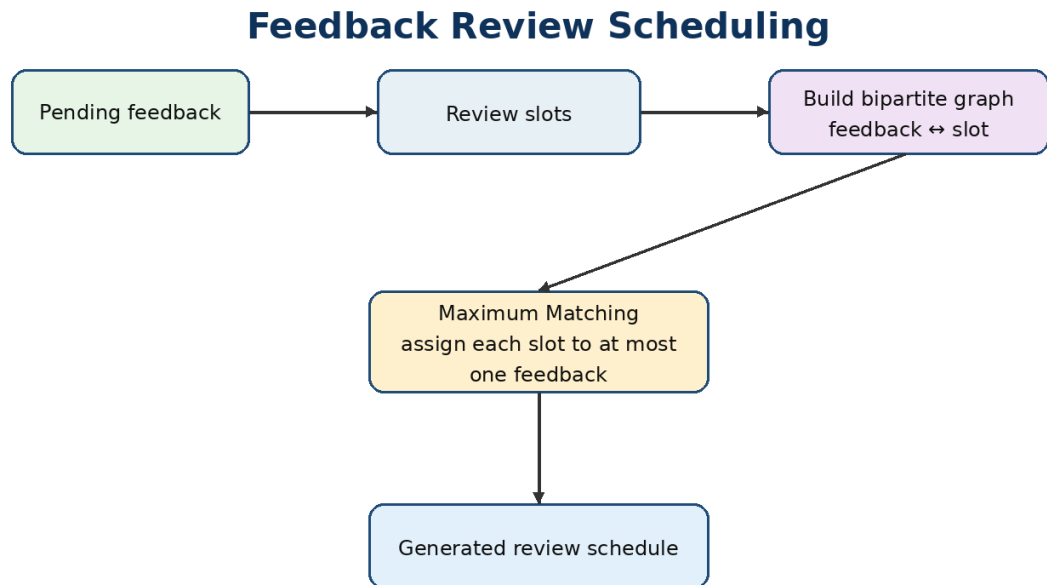


Fig. 4.5 Feedback Review Scheduling

4.8 Random Feedback Audit

The Random Feedback Audit operation accepts a sample size and applies reservoir sampling to the list of feedback files. The first k items are placed in the reservoir. Each subsequent item has a probability-based opportunity to replace an existing sample position. This allows a fixed-size sample to be selected from the complete corpus without requiring the entire sample to be stored in advance.

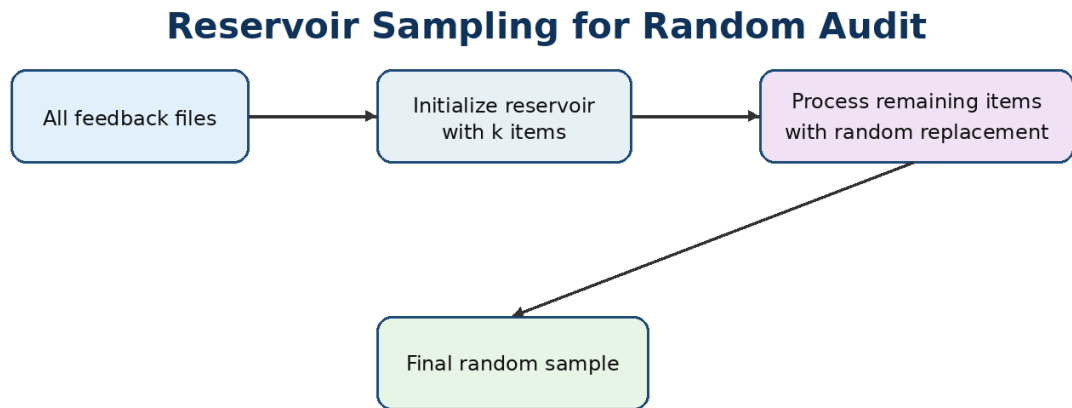


Fig. 4.6 Reservoir Sampling for Random Audit

4.9 File Handling and Corpus Management

ReadFileContents.java discovers files whose names start with Feedback and end with .txt, sorts them numerically by their feedback number, and provides readFile() and writeFile() methods. StudentModule uses nextId() to find the largest existing feedback ID and create the next one. Feedback IDs are presented to the user as numbers even though the physical records are text files such as Feedback01.txt.

4.10 Pseudocode

4.10.1 Feedback Submission

```

START
Read feedback text
IF feedback is empty
    Display "Feedback cannot be empty"
    STOP
END IF

For every feedback file
    Compute Levenshtein similarity
    Keep the highest similarity and its file
END FOR

IF highest similarity >= 60%
    Display existing feedback
    Ask whether to upvote
    IF yes, record vote after duplicate check
ELSE

```

```

        Generate next Feedback ID
        Write new feedback to FeedbackXX.txt
    END IF
END

```

4.10.2 Keyword Search

```

START
Read keyword
IF keyword is empty
    STOP
END IF
For every feedback file
    Read feedback text
    Run KMP search(text, keyword)
    IF match found
        Display Feedback ID, votes, and text
    END IF
END FOR
IF no match found
    Display "No feedback found."
END IF
END

```

4.10.3 Voting

```

START
Read Feedback ID
Read Student ID
IF feedback file does not exist
    Display "Feedback ID not found."
    STOP
END IF
Scan votes.txt
IF Student ID + Feedback ID pair already exists
    Display "You already voted for this feedback."
ELSE
    Append Student ID|FeedbackXX.txt to votes.txt
    Display updated vote count
END IF
END

```

4.10.4 Review Scheduling

```

START
Collect all Pending feedback
Read review slot count and slot names
Create bipartite graph: feedback -> slot
Run maximum matching
Print matched feedback and slots
Print number of scheduled feedback records
END

```

4.10.5 Random Audit

```

START
Read all feedback files
Read sample size k
Initialize reservoir with first k items
For each remaining item
    Generate random index
    If index < k, replace that reservoir position

```

```
END FOR
Display reservoir sample
END
```

4.11 Error Handling and Data Integrity

- Empty feedback is rejected.
- Empty Student ID is rejected during voting.
- Invalid feedback IDs are rejected.
- Voting is rejected when the Student ID has already voted for the same feedback.
- Invalid status choices are rejected.
- Invalid review-slot counts are rejected.
- Invalid random-audit sample sizes are rejected.
- File read/write failures produce a clear terminal message rather than silently continuing the requested operation.

The system also maintains data integrity by keeping each feedback record in its own file, storing votes as Student ID–Feedback pairs, and storing status separately from feedback text. The current implementation uses plain text files, so the integrity of those files depends on the local file system and the correctness of the program operations.

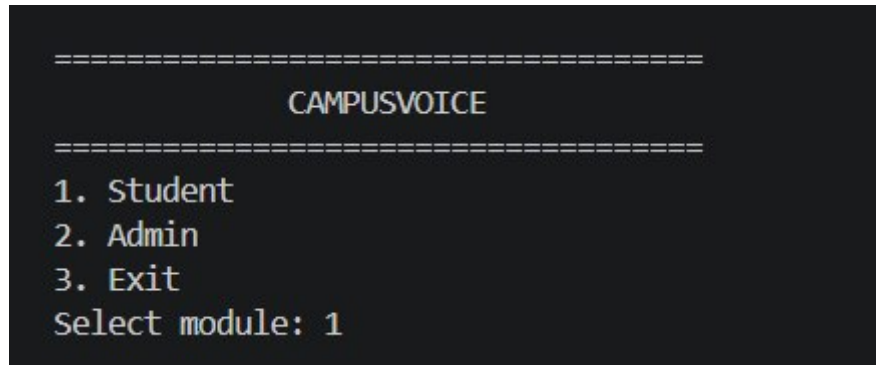
5. RESULTS AND TESTING

The Campus Voice program was compiled and executed successfully using the Java compiler. The core command used for the project is:

```
javac -encoding UTF-8 -d out Main.java admin/*.java student/*.java
algorithms/*.java
java -cp out Main
```

The following results are based on the implemented menu flow and representative terminal tests. The project corpus supplied with the implementation contains 99 feedback records and the initial votes.txt file is empty, while status.txt contains the feedback-status entries.

5.1 Main Menu Execution



```
=====
                        CAMPUSVOICE
=====
1. Student
2. Admin
3. Exit
Select module: 1
```

Fig. 5.1 Representative Terminal Execution – Main Menu

The main menu correctly presents Student, Admin, and Exit options. Selecting Student or Admin transfers control to the corresponding module, while Exit terminates the program.

5.2 Student Module Execution

The Student Module provides three active operations: Submit Feedback, Top Voted Feedback, and Search Feedback. The menu remains active until Back is selected. Empty feedback is rejected before similarity processing begins.

5.3 Similar Feedback Detection

```
===== STUDENT MODULE =====
1. Submit Feedback
2. Top Voted Feedback
3. Search Feedback
4. Back
Choice: 1

Enter feedback: Attendance requests should have clear response

Highest similarity: 67.65%

SIMILAR FEEDBACK FOUND
-----
Feedback ID: 95
Similarity: 67.65%

Existing feedback:
Attendance correction requests should have a clear response timeline.

Would you like to upvote this feedback? (y/n): y
Enter Student ID: 30008
Vote recorded. Total votes: 1
```

Fig. 5.2 Representative Terminal Execution – Duplicate Detection

A representative test used a feedback sentence similar to Feedback ID 1. The comparison produced a similarity above the 60% threshold, so the program displayed the existing Feedback ID and offered the user an upvote instead of creating another record. This confirms the duplicate-recognition workflow.

5.4 Keyword Search

```
===== STUDENT MODULE =====
1. Submit Feedback
2. Top Voted Feedback
3. Search Feedback
4. Back
Choice: 3

Enter keyword: attendance

Feedback ID: 16 | Votes: 0
Attendance information should be updated regularly. Students should be able to report attendance errors easily.

Feedback ID: 95 | Votes: 1
Attendance correction requests should have a clear response timeline.
```

Fig. 5.3 Representative Terminal Execution – Search

The keyword search was tested with the keyword WiFi. The KMP implementation found matching records and displayed their Feedback IDs, vote counts, and stored text. A keyword that does not occur in the corpus produces the message “No feedback found.”

5.5 Voting

Test Condition	Expected Result	Result
Valid Feedback ID + Student ID	Vote is appended to votes.txt	Passed
Same Student ID votes for same Feedback ID again	Duplicate vote is rejected	Passed
Invalid Feedback ID	Operation is rejected	Passed
Empty Student ID	Operation is rejected	Passed

5.6 Administrator Operations

Operation	Test	Expected Result	Result
View All Feedback	Open Admin → View All Feedback	All records show ID, votes, status and text	Passed
Status Update	Change a valid record to In Progress/Completed	status.txt updated	Passed
Priority Analysis	Enter a valid issue limit	Priority issues printed	Passed
Review Schedule	Enter valid slots	Maximum matching assignment printed	Passed
Random Audit	Enter sample size k	k feedback IDs sampled	Passed

5.7 Overall Test Results

S.No.	Functionality	Result
1	Main Menu Navigation	Passed
2	Student Module Menu	Passed
3	Empty Feedback Validation	Passed
4	Levenshtein Similarity Detection	Passed
5	New Feedback File Creation	Passed
6	Top Voted Feedback	Passed
7	KMP Keyword Search	Passed
8	Voting	Passed
9	Duplicate Vote Prevention	Passed
10	Admin Feedback Viewing	Passed
11	Feedback Status Update	Passed
12	Greedy Priority Analysis	Passed
13	Bipartite Review Scheduling	Passed
14	Reservoir Sampling Audit	Passed

6. CHALLENGES AND LIMITATIONS

6.1 Challenges Faced

- Detecting similar feedback when students use different wording.
- Maintaining unique feedback IDs while using individual text files.
- Searching a growing corpus efficiently.
- Preventing duplicate votes using Student ID and Feedback ID.
- Representing administrative priority using student support.
- Assigning multiple pending feedback records to a limited number of review slots.
- Selecting a representative random sample from the feedback corpus.
- Keeping algorithmic logic separate from console and file-management code.

6.2 Solutions Applied

Challenge	Solution
Different wording for the same concern	Levenshtein similarity with a 60% threshold.
Unique feedback identity	Numeric ID derived from FeedbackXX.txt and nextId() generation.
Repeated keyword scanning	KMP pattern matching with an LPS array.
Repeated voting	Scan votes.txt for the same Student ID and feedback file.
Limited administrative capacity	Greedy maximum coverage priority selection.
Limited review slots	Bipartite maximum matching.
Random audit of a large list	Reservoir sampling.

6.3 Limitations

- The application is terminal-based and does not provide a graphical user interface.
- There is no backend server or API layer in the current implementation.
- There is no relational database; persistence is file-based.
- The similarity mechanism is based on normalized text and edit distance, so semantically similar but differently worded feedback may not always cross the 60% threshold.
- KMP search is keyword-based and does not perform semantic search.

- The voting mechanism relies on the Student ID entered by the user; there is no separate identity-management or authentication system.
- The current review scheduler connects every pending feedback to every slot, so it demonstrates maximum matching rather than a preference-aware scheduling model.
- The usefulness of priority analysis depends on the voting data available in votes.txt.

7. CONCLUSION AND FUTURE WORK

7.1 Conclusion

Campus Voice successfully demonstrates a terminal-based student feedback and voting system implemented in Java. The project provides a practical workflow for submitting feedback, detecting closely related concerns, searching the corpus, voting for existing issues, viewing feedback, updating status, prioritizing issues, scheduling reviews, and auditing a random sample.

The major strength of the project is the direct integration of DSA techniques into the application. Levenshtein distance is used to compare feedback, KMP is used for keyword search, greedy maximum coverage supports priority analysis, bipartite matching supports review scheduling, and reservoir sampling supports random auditing. These algorithms are not presented only as theoretical examples; they are connected to concrete functions in the Student and Admin modules.

The final implementation remains intentionally lightweight. It uses the Java console and local text files instead of a web interface, backend service, API, or database. This matches the current project implementation and keeps the focus on DSA-based problem solving and modular Java programming.

7.2 Future Work

- Improve similarity detection with semantic or embedding-based techniques while keeping the current edit-distance method as a baseline.
- Add categories such as academics, infrastructure, hostel, transport, library, and connectivity.
- Provide richer priority scoring using votes, status, category, and age of feedback.
- Add a preference-aware review scheduler rather than connecting every feedback to every slot.
- Add audit history and more detailed administrative reports.
- Add controlled user authentication if the project is later extended beyond the current terminal prototype.

- If required in a future version, a GUI or web interface can be added as a separate presentation layer without changing the core algorithms.
- If required later, file storage can be migrated to a database, but this is outside the current implementation.

7.3 Skills Learned During Development

- Java programming and package-based modular design.
- File handling using java.io and java.nio.file.
- Implementation and application of string-matching algorithms.
- Dynamic programming through Levenshtein edit distance.
- Graph algorithms through bipartite matching.
- Greedy algorithm design for maximum-coverage style prioritization.
- Randomized algorithms through reservoir sampling.
- Console input validation and menu-driven application design.
- Testing, debugging, and handling file I/O errors.
- Technical documentation and mapping DSA concepts to practical requirements.

REFERENCES

- [1] I. Sommerville, Software Engineering, 10th ed., Pearson Education, 2016.
- [2] R. S. Pressman and R. B. Maxim, Software Engineering: A Practitioner's Approach, 9th ed., McGraw-Hill Education, 2019.
- [3] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 4th ed., MIT Press, 2022.
- [4] R. W. Hamming, Coding and Information Theory, Prentice Hall, 1980.
- [5] Project source code: CampusVoice, Java source files supplied with the project archive, 2026–2027.
- [6] Project README: CAMPUSVOICE – Terminal-based campus feedback platform, supplied with the project archive.

APPENDIX A – SETUP AND EXECUTION

A.1 Requirements

- Java Development Kit (JDK) compatible with the source syntax.
- A terminal/command prompt.
- The complete CampusVoice project folder with corpus and data directories.

A.2 Project Structure

```
CampusVoice/  
├── Main.java  
├── admin/  
│   └── AdminModule.java  
├── student/  
│   └── StudentModule.java  
├── algorithms/  
│   ├── BipartiteMatching.java  
│   ├── KMPSearch.java  
│   ├── LevenshteinDistance.java  
│   ├── PriorityOptimizer.java  
│   ├── RandomizedAlgorithms.java  
│   └── ReadFileContents.java  
├── corpus/  
│   └── FeedbackXX.txt files  
├── data/  
│   ├── votes.txt  
│   └── status.txt  
└── README.txt
```

A.3 Compilation

```
cd CampusVoice  
javac -encoding UTF-8 -d out Main.java admin/*.java student/*.java  
algorithms/*.java
```

A.4 Execution

```
java -cp out Main
```

A.5 Usage

1. Choose 1 for Student or 2 for Admin from the main menu.
2. Follow the displayed terminal prompts.
3. Use Back to return to the main menu.
4. Choose Exit to terminate the program.

No web server, API service, database server, or browser setup is required for the current project.

APPENDIX B – IMPORTANT CODE SNIPPETS

The following snippets are selected directly from the supplied project source code. Line ranges are included so that the corresponding implementation can be located easily in the project files.

B.1 Main Menu – Main.java (Lines 5–21)

```
public class Main {
    public static void main(String[] args){
        Scanner sc=new Scanner(System.in);
        while(true){

System.out.println("\n=====");
                System.out.println("                CAMPUSVOICE");

System.out.println("=====");
                System.out.println("1. Student");
                System.out.println("2. Admin");
                System.out.println("3. Exit");
                System.out.print("Select module: ");
                switch(sc.nextLine().trim()){
                    case"1"->new StudentModule(sc).start();
                    case"2"->new AdminModule(sc).start();
                    case"3"-> {System.out.println("Thank you for using
CampusVoice.");return;}
                    default->System.out.println("Invalid choice.");
                }
            }
        }
    }
}
```

B.2 Similarity Detection – StudentModule.java (Lines 48–94)

```
File best = null;
double bestScore = 0;

for (File f : reader.getCorpusFiles(CORPUS)) {
    double score = edit.similarity(input, reader.readFile(f));
    if (score > bestScore) {
        bestScore = score;
        best = f;
    }
}

if (best != null && bestScore >= THRESHOLD) {
    int id = extractId(best);
    System.out.println("SIMILAR FEEDBACK FOUND");
    System.out.println("Feedback ID: " + id);
    // display existing feedback and optionally vote
} else {
    int id = nextId();
    File f = feedbackFile(id);
    reader.writeFile(f, input);
}
```

B.3 Voting Validation – StudentModule.java (Lines 160–195)

```
for (String line : Files.readAllLines(Path.of(VOTES))) {
    String[] p = line.split("\\|", -1);
    if (p.length == 2
        && p[0].equalsIgnoreCase(student)
        && p[1].equalsIgnoreCase(file.getName())) {
        System.out.println("You already voted for this feedback.");
        return;
    }
}

Files.writeString(
    Path.of(VOTES),
    student + "|" + file.getName() + System.lineSeparator(),
    StandardOpenOption.CREATE,
    StandardOpenOption.APPEND
);
```

B.4 Levenshtein Similarity – LevenshteinDistance.java (Lines 42–62)

```
public double similarity(String first, String second) {
    String a = normalize(first);
    String b = normalize(second);
    int maxLength = Math.max(a.length(), b.length());
    if (maxLength == 0) {
        return 1.0;
    }
    int distance = calculate(a, b);
    return 1.0 - ((double) distance / maxLength);
}
```

B.5 KMP Search – KMPSearch.java (Lines 25–50)

```
public boolean search(String text, String pattern) {
    if (text == null || pattern == null || pattern.isEmpty()) {
        return false;
    }
    text = text.toLowerCase();
    pattern = pattern.toLowerCase();
    int[] lps = buildLPS(pattern);
    int i = 0;
    int j = 0;
    while (i < text.length()) {
        if (text.charAt(i) == pattern.charAt(j)) {
            i++; j++;
            if (j == pattern.length()) return true;
        } else if (j > 0) {
            j = lps[j - 1];
        } else {
            i++;
        }
    }
    return false;
}
```

B.6 Greedy Priority Selection – PriorityOptimizer.java (Lines 31–65)

```
while (!remaining.isEmpty() && selected.size() < limit) {
    Issue best = null;
    int bestGain = -1;
    for (Issue issue : remaining) {
        int gain = 0;
        for (String student : issue.getStudents()) {
            if (!coveredStudents.contains(student)) gain++;
        }
        if (gain > bestGain) {
            best = issue;
            bestGain = gain;
        }
    }
    if (best == null) break;
    selected.add(best);
    coveredStudents.addAll(best.getStudents());
    remaining.remove(best);
}
```

B.7 Bipartite Matching – BipartiteMatching.java (Lines 26–47)

```
public int maximumMatching() {
    int matches = 0;
    for (int feedback = 0; feedback < feedbackCount; feedback++) {
        boolean[] visited = new boolean[slotCount];
        if (findMatch(feedback, visited)) matches++;
    }
    return matches;
}

private boolean findMatch(int feedback, boolean[] visited) {
    for (int slot = 0; slot < slotCount; slot++) {
        if (!graph[feedback][slot] || visited[slot]) continue;
        visited[slot] = true;
        if (matchedFeedback[slot] == -1 ||
            findMatch(matchedFeedback[slot], visited)) {
            matchedFeedback[slot] = feedback;
            return true;
        }
    }
    return false;
}
```

B.8 Reservoir Sampling – RandomizedAlgorithms.java (Lines 7–30)

```
public <T> List<T> reservoirSample(List<T> items, int sampleSize) {
    if (sampleSize <= 0 || items.isEmpty()) {
        return new ArrayList<>();
    }
    sampleSize = Math.min(sampleSize, items.size());
    List<T> reservoir = new ArrayList<>();
    for (int i = 0; i < sampleSize; i++) {
        reservoir.add(items.get(i));
    }
}
```

```
Random random = new Random();
for (int i = sampleSize; i < items.size(); i++) {
    int j = random.nextInt(i + 1);
    if (j < sampleSize) reservoir.set(j, items.get(i));
}
return reservoir;
}
```